Module 10: Packages

Duration

1 Lecture + 2–3 Laboratory Hours

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of packages in PL/SQL.
- Create package specifications and package bodies.
- Differentiate between package specification and package body.
- Organize related procedures, functions, variables, and cursors into reusable modules.
- Develop modular and maintainable database applications using packages.
- Apply packages in real-world enterprise applications.

10.1 Introduction to Packages

What is a Package?

A **Package** is a database object in Oracle that groups together related **procedures, functions, variables, cursors, exceptions, and constants** into a single unit.

A package helps organize PL/SQL code into logical modules, making applications easier to develop, maintain, and reuse.

A package consists of **two parts**:

1. **Package Specification**
2. **Package Body**

Real-Life Analogy

Consider a **Smartphone**.

The mobile phone has different applications such as:

- Camera
- Calculator
- Contacts
- Gallery
- Messages

Each application contains multiple related functions.

Similarly, a **PL/SQL Package** groups related database operations into a single module.

Example:

Employee Package

- Add Employee
- Delete Employee
- Update Salary
- Search Employee
- Display Employee

All these related operations are grouped inside one package.

Why Do We Need Packages?

Suppose a company has hundreds of procedures.

Without packages:

```
Procedure 1
Procedure 2
Procedure 3
Procedure 4
Procedure 5
...

100 Procedures
```

Managing them becomes difficult.

Using Packages

```
Employee Package
```

```
    Add Employee
    Delete Employee
    Search Employee
    Update Salary

Student Package

    Add Student
    Update Marks
    Display Result

Bank Package

    Deposit
    Withdraw
    Balance Inquiry

Library Package

    Issue Book
    Return Book
```

Everything becomes organized.

Components of a Package

A package has two parts.

```
Package
│
├── Package Specification
│       Public Variables
│       Procedures
│       Functions
│       Cursors
│
└── Package Body
        Actual Implementation
        Business Logic
```

10.2 Package Specification

Definition

The **Package Specification** is the **interface** of the package.

It contains declarations of:

- Procedures
- Functions
- Variables
- Constants
- Cursors
- Exceptions

The specification tells Oracle **what** is available.

It does **not** contain implementation.

Syntax

```
CREATE OR REPLACE PACKAGE package_name

IS

    variable datatype;

    PROCEDURE procedure_name(...);

    FUNCTION function_name(...)

    RETURN datatype;

END package_name;
/
```

Example

```
CREATE OR REPLACE PACKAGE employee_pkg

IS

    company_name VARCHAR2(50):='ABC Technologies';

    PROCEDURE display_company;
```

```
    FUNCTION annual_salary

    (
        salary NUMBER
    )

    RETURN NUMBER;

END employee_pkg;
/
```

Explanation

Here,

The package specification declares

- Variable
- Procedure
- Function

but **does not define how they work.**

10.3 Package Body

Definition

The Package Body contains the **implementation** of all procedures and functions declared in the specification.

It defines **how the operations are performed.**

Syntax

```
CREATE OR REPLACE PACKAGE BODY package_name

IS

    Procedure Definition
```

```
    Function Definition

END package_name;
/
```

Example

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE BODY employee_pkg

IS

PROCEDURE display_company

IS

BEGIN

DBMS_OUTPUT.PUT_LINE(company_name);

END;

FUNCTION annual_salary

(
salary NUMBER
)

RETURN NUMBER

IS

BEGIN

RETURN salary*12;

END;

END employee_pkg;
/
```

Executing Package Members

```
BEGIN

employee_pkg.display_company;

END;
/
```

Output

```
ABC Technologies
```

Using Function

```
SELECT

employee_pkg.annual_salary(50000)

FROM dual;
```

Output

```
600000
```

Package Initialization

A package may also contain an initialization block.

Example

```
BEGIN

DBMS_OUTPUT.PUT_LINE
(
'Package Loaded Successfully'
);

END;
```

This block executes **only once** when the package is first referenced during a session.

Advantages of Packages

Packages are widely used because they provide several benefits.

1. Code Reusability

One package can be used by multiple applications.

---

2. Better Organization

Related procedures and functions remain together.

---

3. Improved Security

Only the package specification is visible to users.

Implementation remains hidden inside the package body.

---

4. Easier Maintenance

Updating package body does not require changing applications if the specification remains unchanged.

---

5. Better Performance

Oracle loads the entire package into memory during the first call.

Subsequent calls execute faster.

---

6. Modular Programming

Large projects become easier to manage.

---

7. Encapsulation

Internal implementation remains hidden.

---

8. Reduced Network Traffic

Instead of multiple SQL calls, one package call performs several operations.

Difference between Procedure and Package

Procedure	Package
Single program	Collection of programs
Performs one task	Performs multiple related tasks
Easier for small applications	Better for enterprise applications
Difficult to organize many procedures	Better organization
Limited reusability	Highly reusable

Real-Time Applications

Packages are used extensively in:

- Banking Systems
- Hospital Management
- Payroll Systems
- Railway Reservation
- Insurance Applications
- ERP Solutions
- Human Resource Management
- Student Information Systems
- Library Management Systems
- E-Commerce Applications

Summary

A **Package** is a collection of related PL/SQL program units such as procedures, functions, variables, cursors, constants, and exceptions. It consists of a **Package Specification**, which declares the public interface, and a **Package Body**, which implements the business logic. Packages improve modularity, reusability, security, maintainability, and performance, making them an essential feature for developing large-scale Oracle database applications.

Practical Program 1: Employee Package

Aim

To create an Employee Package for displaying employee details and calculating annual salary.

Step 1: Create Employee Table

```
CREATE TABLE employee
(
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(40),
    department VARCHAR2(30),
    salary NUMBER
);

INSERT INTO employee VALUES (101,'Rahul','IT',45000);
INSERT INTO employee VALUES (102,'Priya','HR',55000);

COMMIT;
```

Step 2: Package Specification

```
CREATE OR REPLACE PACKAGE employee_pkg

IS

PROCEDURE display_employee

(
p_emp_id NUMBER
);

FUNCTION annual_salary

(
salary NUMBER
)

RETURN NUMBER;

END employee_pkg;
/
```

Step 3: Package Body

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE BODY employee_pkg

IS
```

```
PROCEDURE display_employee

(
p_emp_id NUMBER
)

IS

v_name employee.emp_name%TYPE;
v_dept employee.department%TYPE;
v_salary employee.salary%TYPE;

BEGIN

SELECT emp_name,department,salary

INTO v_name,v_dept,v_salary

FROM employee

WHERE emp_id=p_emp_id;

DBMS_OUTPUT.PUT_LINE('Employee ID   : '||p_emp_id);
DBMS_OUTPUT.PUT_LINE('Employee Name : '||v_name);
DBMS_OUTPUT.PUT_LINE('Department    : '||v_dept);
DBMS_OUTPUT.PUT_LINE('Salary        : '||v_salary);

END;

FUNCTION annual_salary

(
salary NUMBER
)

RETURN NUMBER

IS

BEGIN

RETURN salary*12;

END;

END employee_pkg;
/
```

Step 4: Execute Package

```
BEGIN

employee_pkg.display_employee(101);

END;
```

```
/
```

Output

```
Employee ID   : 101
Employee Name : Rahul
Department    : IT
Salary        : 45000
```

Execute Function

```
SELECT employee_pkg.annual_salary(45000)

FROM dual;
```

Output

```
540000
```

Practical Program 2: Student Package

Aim

To create a package for calculating grades and displaying student details.

Step 1: Create Student Table

```
CREATE TABLE student
(
roll_no NUMBER PRIMARY KEY,
student_name VARCHAR2(40),
course VARCHAR2(30),
marks NUMBER
);

INSERT INTO student VALUES(1,'Rahul','B.Tech CSE',85);
INSERT INTO student VALUES(2,'Priya','B.Tech IT',72);

COMMIT;
```

Step 2: Package Specification

```
CREATE OR REPLACE PACKAGE student_pkg
```

```
IS

PROCEDURE display_student

(
p_roll NUMBER
);

FUNCTION calculate_grade

(
marks NUMBER
)

RETURN VARCHAR2;

END student_pkg;
/
```

Step 3: Package Body

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE BODY student_pkg

IS

PROCEDURE display_student

(
p_roll NUMBER
)

IS

v_name student.student_name%TYPE;
v_course student.course%TYPE;
v_marks student.marks%TYPE;

BEGIN

SELECT student_name,course,marks

INTO v_name,v_course,v_marks

FROM student

WHERE roll_no=p_roll;

DBMS_OUTPUT.PUT_LINE('Roll Number : '||p_roll);
DBMS_OUTPUT.PUT_LINE('Name        : '||v_name);
DBMS_OUTPUT.PUT_LINE('Course      : '||v_course);
DBMS_OUTPUT.PUT_LINE('Marks       : '||v_marks);
```

```
END;

FUNCTION calculate_grade

(
marks NUMBER
)

RETURN VARCHAR2

IS

BEGIN

IF marks>=90 THEN
RETURN 'A+';

ELSIF marks>=80 THEN
RETURN 'A';

ELSIF marks>=70 THEN
RETURN 'B';

ELSIF marks>=60 THEN
RETURN 'C';

ELSE
RETURN 'F';

END IF;

END;

END student_pkg;
/
```

Step 4: Execute Package

```
BEGIN

student_pkg.display_student(1);

END;
/
```

Output

```
Roll Number : 1
Name        : Rahul
Course      : B.Tech CSE
Marks       : 85
```

Execute Function

```
SELECT student_pkg.calculate_grade(85)

FROM dual;
```

Output

```
A
```

Additional Practice Programs

1. Bank Package (Deposit, Withdrawal, Balance Inquiry)
2. Library Package (Issue Book, Return Book, Search Book)
3. Payroll Package (Salary, Bonus, Tax Calculation)
4. Department Package (Add, Update, Delete Department)
5. Inventory Package (Add Product, Update Stock)
6. Hospital Package (Patient Registration, Appointment)
7. Insurance Package (Premium Calculation)
8. Electricity Bill Package
9. GST Management Package
10. Examination Result Package

Viva Questions

1. What is a package in PL/SQL?
2. What are the two components of a package?
3. What is the difference between a package specification and a package body?
4. Why are packages preferred in enterprise applications?
5. Can a package contain variables and cursors?
6. What are the advantages of packages over standalone procedures?
7. What is package initialization?
8. Can a package contain multiple procedures and functions?
9. How does Oracle improve performance when using packages?
10. Explain encapsulation in the context of PL/SQL packages.